Lookking Technical Deep Dive

Student: Karim Akoudad

Brief: UIR S8 Integrated Project - Building Multi-Agent AI Systems

Purpose: Defense preparation notes for pipeline, model training, hyperparameters, and honest limitations.

Date: May 18, 2026

1. What The Professor Asked For

The official brief requires a multi-agent AI system where specialized agents collaborate to solve a real-world problem. At least one agent must use a deep learning model trained by us as a real functional tool, not decoration.

Minimum requirements from the brief:

```
| Requirement | Our Status | Honest Comment |
|---|---:|---|
| 2 specialists + 1 orchestrator | Done | We have Orchestrator, Discovery, and Scoring agents. |
| Distinct agent roles and tools | Done | Discovery has search tools; Scoring has DL scorer; Orchestrator parses intent. |
| PyTorch DL model trained/fine-tuned by us | Done | DistilBERT classifier trained in `model/train.py`. |
| Evaluation with accuracy/confusion matrix | Done | `model/metrics.json` and `model/confusion_matrix.png`. |
| At least 2 tools including DL model | Done | `search_places`, `search_leads`, `score_match`. |
| Human-in-the-loop checkpoint | Mostly done | User chooses mode and can approve/refine results. Approval is logged but not saved
| Error handling | Partially done | Try/except, setup checks, API fallback exist. Unexpected LLM output handling is still basic. |
| Logging with timestamps in JSON | Done | `logs/agent_logs.json`, through `utils/logger.py`. |
| README setup instructions | Done | `README.md` now explains setup, training, and running. |
| GitHub repo | Missing | This folder is not a git repo yet. |
| PDF report, slides, demo video | In progress | Reports exist; slides/video still need final creation. |
```

Bottom line: the core technical project satisfies the important AI requirements. The weak points are packaging, README/GitHub, final demo materials, and real-world validation.

2. What The App Does

Lookking is a Telegram bot for two tasks:

1. Find places such as restaurants, spas, gyms, hotels, cafes, and barbershops.
2. Find business leads for a freelancer or agency offering a service.

The user flow:

```
/start
  -> choose Place or Leads
  -> type natural-language query
  -> bot runs multi-agent pipeline
  -> bot returns top ranked results
  -> user clicks Done or Add Info
```

Example:

```
[MODE: places] sushi restaurant Rabat open now
```

The system then:

1. Understands the intent.
2. Searches candidates.
3. Scores candidates with the trained model.
4. Returns the top 3 results.

3. The End-To-End Pipeline

The real pipeline is:

```
Telegram user
  -> bot/telegram_bot.py
  -> agents/crew_setup.py
  -> tools/search_tool.py
  -> tools/nominatim_tool.py or CSV data
  -> tools/dl_scorer_tool.py
  -> model/lookking_model.pt
  -> ranked Telegram response
```

Step 1: Telegram Interface

`bot/telegram_bot.py` handles the user interface. It keeps a state per chat:

- `MODE_PICK`
- `AWAITING_QUERY`
- `RESULTS_SHOWN`
- `AWAITING_REFINEMENT`
- `IDLE`

Why this matters:

- It prevents the bot from guessing whether the user wants places or leads.
- It gives the user control through buttons.
- It supports refinement without restarting the whole conversation.

Step 2: Mode Prefix

When the user picks a mode, the bot rewrites the query:

```
[MODE: places] user text
```

or:

```
[MODE: leads] user text
```

This is a practical design choice. The LLM can be ambiguous, so the explicit prefix tells the agents: do not override the user-selected mode.

Step 3: CrewAI Agents

`agents/crew_setup.py` builds a sequential CrewAI pipeline. Sequential means each task runs after the previous one and receives its context.

```
Analyze task -> Discover task -> Score task
```

Step 4: Search Tools

For place search:

- Try live OpenStreetMap/Nominatim first.
- If it fails or returns empty, fall back to `data/places.csv`.

For leads:

- Search `data/leads.csv`.

Step 5: DL Scoring Tool

The scoring tool receives:

```
QUERY: user query | CANDIDATE: candidate description
```

It returns:

- Low Match, Medium Match, or High Match.
- Confidence percentage.
- Class probabilities.

4. The Agents

4.1 Query Orchestrator

Purpose:

- Parse the natural-language query.
- Extract mode, city, category, and constraints.
- Interpret refinement messages.

What it does not do:

- It does not search.
- It does not score candidates.

Defense answer:

> The orchestrator is responsible for understanding the request and creating structure from raw user text. It is the control layer of the system.

4.2 Discovery Specialist

Purpose:

- Retrieve candidate places or leads.
- Use the correct search tool based on the mode.

Tools:

- `Search Places`
- `Search Leads`

Defense answer:

> The discovery agent is the retrieval layer. It does not decide final ranking; it only finds candidate results.

4.3 AI Match Scorer

Purpose:

- Score candidates using the trained deep learning model.
- Rank results.
- Produce the final top 3.

Tool:

- `Score Match`

Defense answer:

> The scoring agent is where the trained model is used as a real tool. It asks the model whether each candidate is a low, medium, or high match.

5. The Data: Honest Explanation

The project uses three datasets:

File	Rows	Real or Synthetic?	Purpose
`data/training_data.csv`	295	Synthetic	Train the matching classifier.
`data/places.csv`	60	Synthetic	Fallback place data if OSM fails.
`data/leads.csv`	62	Synthetic	Business lead search data.

This is important:

> Yes, part of the data is fake/synthetic. It was generated to build and test the pipeline, train the model, and demonstrate the architecture. The live places feature uses OpenStreetMap, but the training data and leads

dataset are synthetic.

The correct defense position:

> I am not claiming that the model is production-ready. I am claiming that the project demonstrates the required architecture: data generation, model training, evaluation, model-as-tool integration, multi-agent orchestration, logging, and HITL.

Why synthetic data was acceptable for this prototype:

- The project time was short.
- We needed labeled query-candidate pairs.
- Public labeled data for this exact Moroccan place/lead matching task is not readily available.
- Synthetic labels let us prove the training and integration pipeline.

What would be needed for production:

- Real user queries.
- Real clicked/approved results.
- Real business lead data.
- A larger evaluation set.
- Human-labeled examples.

6. The Deep Learning Problem

The model solves a classification problem:

```
Input:  user query + candidate result
Output: Low Match / Medium Match / High Match
```

Example:

```
Query: sushi restaurant Rabat open now
Candidate: Tokyo Nights - sushi restaurant in Rabat, open, 4.7 stars
Label: High Match
```

Another example:

```
Query: sushi restaurant Rabat open now
Candidate: Elite Barber - luxury barbershop
Label: Low Match
```

This is not generative AI. It is supervised learning:

- We give the model examples.
- Each example has a correct label.
- The model learns patterns that separate high, medium, and low matches.

7. Why DistilBERT?

We used:

```
distilbert-base-uncased
```

Reasons:

- It is a smaller version of BERT.
- It understands English text semantics better than a simple bag-of-words model.
- It is lighter and faster than full BERT.
- It is suitable for text-pair classification.
- It can run locally on a laptop.

Why not train a transformer from scratch?

> Training from scratch would require huge data and compute. Transfer learning is the correct approach for a small academic project.

Why not use only Gemini?

> The brief requires a trained PyTorch deep learning model. Gemini helps the agents reason, but the DistilBERT model is our trained component.

8. Model Architecture

The architecture is:

```
Input text
  -> DistilBERT tokenizer
  -> DistilBERT encoder
  -> [CLS] embedding, size 768
  -> Dropout
  -> Linear 768 -> 256
  -> ReLU
  -> Dropout
  -> Linear 256 -> 64
  -> ReLU
  -> Dropout
  -> Linear 64 -> 3
  -> Softmax at inference
```

What Is The `[CLS]` Embedding?

Transformer models produce an embedding for each token. The first token is often used as a summary representation of the whole input. We use:

```
out.last_hidden_state[:, 0, :]
```

That gives a 768-dimensional vector representing the query-candidate pair.

Why A Custom Classifier Head?

DistilBERT gives a text representation. The classifier head converts that representation into three classes:

- Low Match
- Medium Match
- High Match

The head is small because the dataset is small. A very large head would overfit even more.

9. Freezing Layers

In `model/model.py`, we freeze:

- Embeddings.
- First 4 of 6 transformer layers.

We train:

- Last 2 transformer layers.
- Custom classifier head.

Why freeze layers?

- The dataset is small.
- Early transformer layers learn general language features.
- If we train too many parameters, the model can memorize the synthetic data.
- Freezing makes training faster and more stable.

Why train the last 2 layers?

- Later transformer layers are more task-specific.
- Fine-tuning them lets the model adapt to the matching task.

Honest limitation:

> Freezing helps, but it does not solve the main limitation: the data is still small and synthetic.

10. Hyperparameters Explained

Batch Size = 16

Meaning:

- The model sees 16 examples before one optimizer update.

Why 16:

- Small enough for local machine memory.
- Large enough to make gradients less noisy than batch size 1 or 4.
- Common value for transformer fine-tuning.

Tradeoff:

- Bigger batch: more stable but more memory.
- Smaller batch: less memory but noisier training.

Epochs = 12

Meaning:

- One epoch means the model has seen the full training set once.
- 12 epochs means 12 passes through the training data.

Why 12:

- Small dataset, so each epoch is fast.
- Enough time for the classifier head to learn.

Honest issue:

> 12 epochs on 295 synthetic examples is likely overfitting. It works for the prototype, but for real data we would use early stopping and a larger validation set.

Max Length = 128

Meaning:

- Every input is padded or truncated to 128 tokens.

Why 128:

- Queries and candidate descriptions are short.
- Faster than 256 or 512.
- Reduces memory use.

Risk:

- Very long candidate descriptions could be truncated.

Dropout = 0.3

Meaning:

- During training, dropout randomly disables 30% of some neural activations.

Why use it:

- It reduces overfitting.
- It forces the classifier not to depend too much on one path.

Why 0.3:

- Common moderate value.
- Stronger than 0.1, useful because dataset is small.

Risk:

- Too much dropout can underfit. Here the model still learned easily because data is simple.

Test Size = 20%

Meaning:

- 80% training data.
- 20% test data.

Why:

- Standard split for small datasets.

Actual numbers:

- 236 training examples.
- 59 test examples.

Limitation:

> 59 test examples is small, and they come from the same synthetic generation process as training examples.

Stratify = Label

Meaning:

- Keep label proportions similar in train and test.

Why:

- Avoid a test set that accidentally has too many high or low examples.
- Important because labels are not perfectly balanced.

Random State = 42

Meaning:

- Makes the train/test split reproducible.

Why:

- If the professor reruns the code, they should get the same split.

11. Why AdamW?

The optimizer updates model weights to reduce the loss.

We used:

```
torch.optim.AdamW(...)
```

Why AdamW is a good choice:

- It is standard for transformer fine-tuning.
- It adapts learning rates per parameter.
- It handles sparse/noisy gradients better than plain SGD.
- It has decoupled weight decay, which is better regularization than classical Adam with L2.

AdamW vs Adam

Adam:

- Adaptive optimizer.
- But weight decay can interact badly with Adam's adaptive updates.

AdamW:

- Same general idea as Adam.
- Weight decay is decoupled from the gradient update.
- Usually preferred for BERT/transformer fine-tuning.

AdamW vs SGD

SGD:

- Simpler.
- Can generalize well but usually needs careful learning-rate scheduling.
- Often slower and less stable for transformer fine-tuning.

For this project:

> AdamW was the pragmatic, standard choice for a small transformer fine-tuning task.

12. Why Two Learning Rates?

The code uses:

```
bert_params lr = 2e-5
head_params lr = 1e-3
```

BERT Learning Rate = 2e-5

The BERT layers are pretrained. We do not want to destroy what they already know.

Why small:

- Transformer weights are sensitive.
- Fine-tuning usually uses small learning rates like 1e-5, 2e-5, or 5e-5.

Classifier Head Learning Rate = 1e-3

The classifier head is new and randomly initialized.

Why larger:

- It starts from zero knowledge.
- It needs to learn faster than the pretrained layers.

Defense answer:

> I used a small learning rate for pretrained transformer layers to preserve language knowledge, and a larger learning rate for the new classifier head because it has to learn from scratch.

13. Why CrossEntropyLoss?

The model predicts one of three classes:

- Low
- Medium
- High

This is multi-class classification. `CrossEntropyLoss` is the standard loss for this type of problem.

It compares:

- The model's raw logits.
- The true class label.

It penalizes the model when it gives low probability to the correct class.

Why not Mean Squared Error?

> MSE is for regression or numeric targets. Here the target is a class, so CrossEntropyLoss is appropriate.

14. Why Gradient Clipping?

The code uses:

```
torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
```

Meaning:

- If gradients become too large, shrink them.

Why:

- Stabilizes training.
- Prevents sudden large updates.
- Common in transformer training.

Defense answer:

> Gradient clipping is a guardrail. It prevents unstable updates, especially when fine-tuning transformer layers.

15. Evaluation: What The 100% Accuracy Really Means

Recorded metrics:

```
Accuracy: 1.0
Confusion matrix:
Low    -> 15 correct
Medium -> 15 correct
High   -> 29 correct
```

This means the model classified all 59 test examples correctly.

But be honest:

> This does not prove production performance. The data is synthetic, small, and generated from repeated templates. The model may have learned patterns from the generator rather than robust real-world semantic matching.

Also important:

> The current training code uses the test set during training to decide the best model. Strictly speaking, that test set acts like a validation set. A more rigorous setup would use train/validation/test splits.

Better future evaluation:

- Train set: learn weights.
- Validation set: choose best epoch and hyperparameters.
- Test set: final untouched evaluation.
- Add real human-labeled examples.

16. What Is Real And What Is Not?

Real:

- Telegram bot implementation.
- CrewAI multi-agent system.
- PyTorch model architecture and training code.
- Saved trained model.
- Live OpenStreetMap/Nominatim place lookup.
- Google Maps and OSM links.
- Logging implementation.
- Human-in-the-loop UI.

Synthetic:

- Training examples.
- Fallback place dataset.
- Lead dataset.
- OSM ratings/prices are synthetic because OSM does not provide review ratings or prices.

Weak/unfinished:

- README exists, but it still needs screenshots/demo polish before GitHub submission.
- No GitHub repo yet.
- No slides/video yet.
- No real user evaluation.
- No robust parser for unexpected LLM output.
- No persistent database for approved results.

This is the most honest defense line:

> The project is a strong academic prototype. It proves the architecture and integration requirements, but it is not yet a production-grade search or lead-generation platform.

17. Error Handling And Robustness

Implemented:

- `main.py` checks that data and model files exist.
- Nominatim errors are caught and logged.
- Place search falls back to CSV.
- `run_lookking` catches exceptions and returns a system error string.
- Telegram handler catches pipeline errors.
- DL scorer handles missing model file and generic exceptions.

Gaps:

- LLM output is not parsed into a strict schema.
- No retry/backoff for Gemini quota errors.
- No tests for edge cases.
- No caching for Nominatim.
- Logs are rewritten as one JSON array, which is fine for a prototype but not scalable.

Defense answer:

> Error handling exists at the tool and app level, but a production system would need stricter structured outputs, retries, caching, unit tests, and better monitoring.

18. Human-In-The-Loop: Strong Or Weak?

Implemented HITL:

- User selects mode.
- User reviews results.
- User clicks Done or Add Info.
- Add Info reruns the search with refinement.

Honest view:

> HITL exists, but it is lightweight. The approval is logged and closes the session; it does not yet save approved results to a database. For the brief, it demonstrates a human checkpoint, but a stronger version would persist approvals and use them as training feedback.

19. How To Explain The Whole System In 60 Seconds

Use this answer:

> Lookking is a Telegram-based multi-agent AI assistant. The user chooses whether they want places or leads, then writes a natural-language query. The orchestrator agent extracts the intent, the discovery agent searches candidates using OpenStreetMap or CSV data, and the scoring agent uses our trained DistilBERT model to rank the candidates as low, medium, or high match. The app includes human-in-the-loop refinement, JSON logging, error handling, and a PyTorch model trained on query-candidate examples.

20. Questions You Should Be Ready For

Why did you choose this project?

Because it naturally decomposes into agents: understanding the query, retrieving candidates, and scoring matches. It also allows a real trained model to be used as a tool.

Is the model actually used?

Yes. The scoring agent calls `score_match`, which loads `model/lookking_model.pt` and classifies query-candidate pairs.

Is the data real?

Partially. OpenStreetMap search is real. Training data, fallback places, and leads are synthetic. Ratings/prices for OSM results are synthetic estimates.

Why did you use Gemini?

Gemini is used as the LLM backend for agent reasoning because the brief allowed Gemini free API or Ollama. The trained PyTorch model is separate.

Why not just ask Gemini to rank results?

Because the brief requires a trained deep learning model as a functional tool. Also, using a local classifier gives a controlled scoring component.

Why did you freeze BERT layers?

To reduce overfitting and training cost because the dataset is small. Earlier BERT layers already contain useful language features.

Why 100% accuracy?

Because the test set is small and synthetic. It shows the prototype works, not that the model is production-ready.

Did you satisfy the brief?

The technical AI requirements are mostly satisfied. Missing submission items are README, GitHub repo, slides, and demo video. Error handling/HITL are present but can be strengthened.

21. What To Improve Before Defense

Highest priority:

1. Stop old bot processes and run one clean live demo.
2. Record a 3-5 minute demo video.
3. Prepare 10-15 slides.
4. Push code to GitHub without `.env`.
5. Add screenshots/demo outputs to `README.md` if time allows.

Technical improvements if time remains:

1. Add a separate validation split.
2. Save classification report to JSON.
3. Add tests for tools and scorer.
4. Add persistent saved approvals.
5. Add Gemini quota fallback or Ollama local fallback.
6. Add real labeled examples from manual testing.

22. Final Honest Position

Do not oversell the project. Say this:

> Lookking is an academic prototype that demonstrates the full required AI architecture: multi-agent orchestration, PyTorch model training, model-as-tool integration, external API search, logging, and human-in-the-loop refinement. The main limitation is that the model training data and leads data are synthetic, so the evaluation proves the pipeline more than real-world performance. The next step would be collecting real user interactions and retraining/evaluating on real labels.

Appendix: Model Confusion Matrix